

中山大学网络空间安全学院必修课程

程序设计I

C Programming

岳 晟

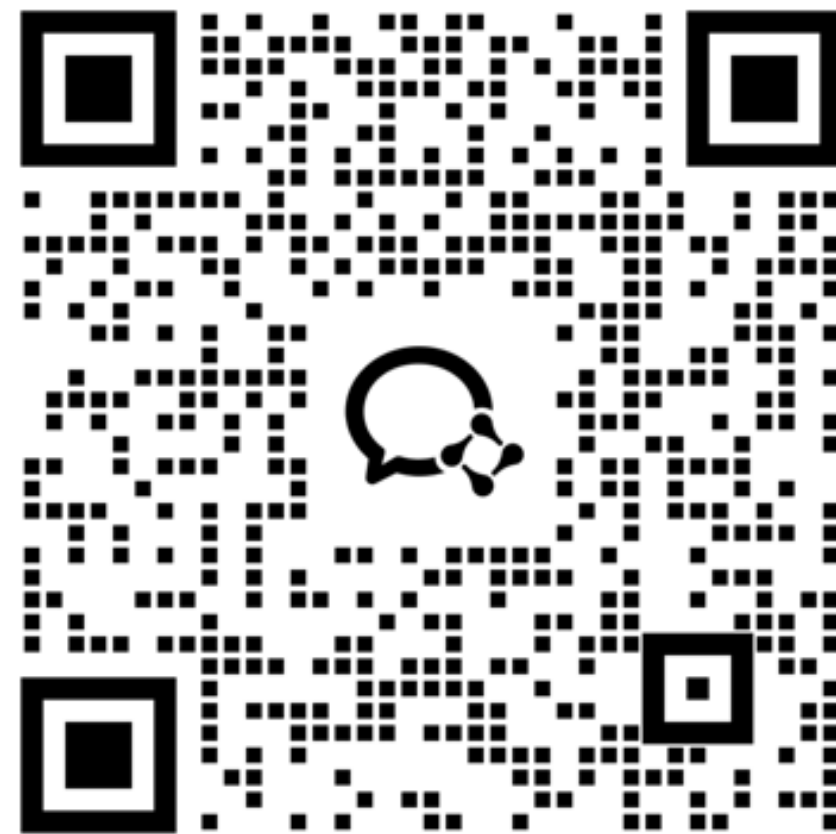
网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2026年秋季

课程信息

- 教师：岳晟
- 联系方式：企业微信 / yuesh5@mail.sysu.edu.cn
- 课程助教：龙婷婷 (博士一年级)，容镇成 (硕士二年级)
- 课程群：



课程信息

□ 教材

- 谭浩强, C程序设计 (第六版)

□ 参考书

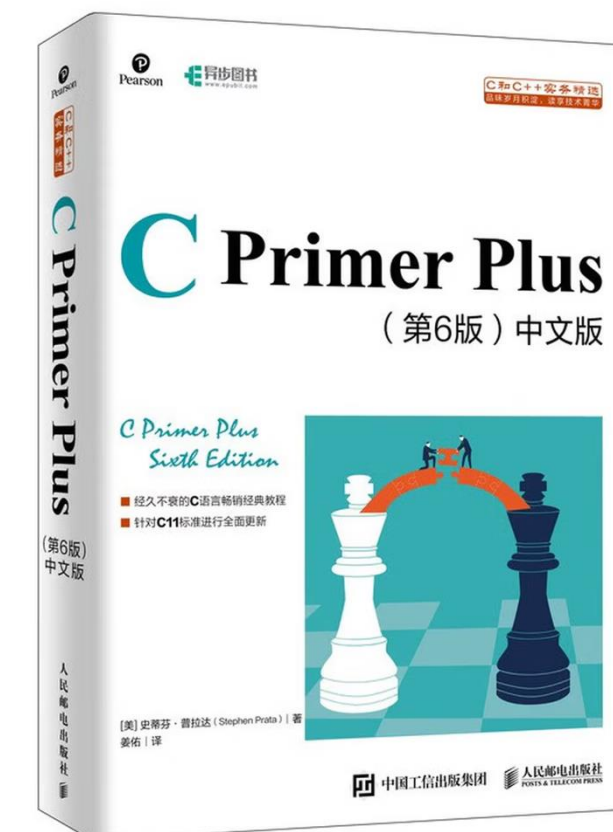
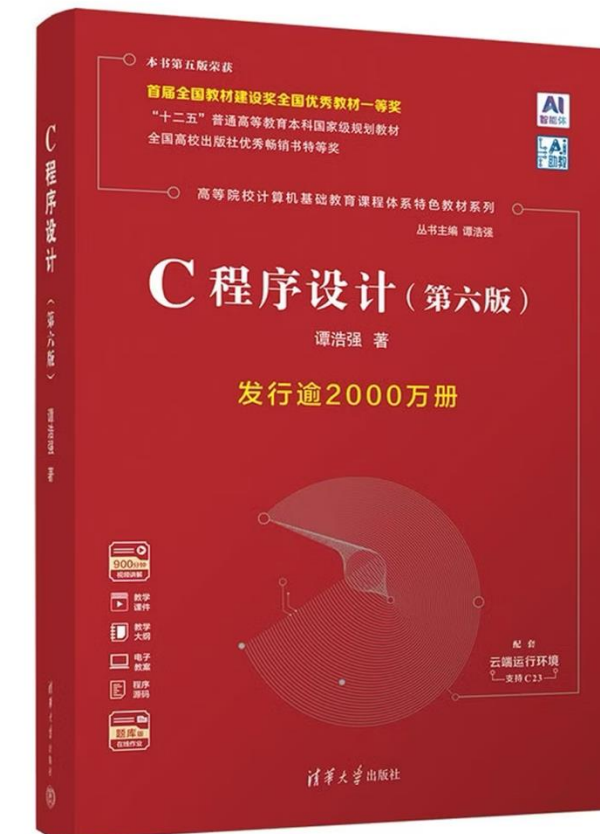
- Stephen Prata, C Primer Plus (第六版)

□ 课堂直播/回放 (深圳校区)

- 腾讯会议

□ 上课时间

- 理论: 1-17周一3-4节、1-9周三3-4节
- 实验: 1-17周一7-8节



课程信息

□ 考核标准 (理论课和实验课成绩相同)

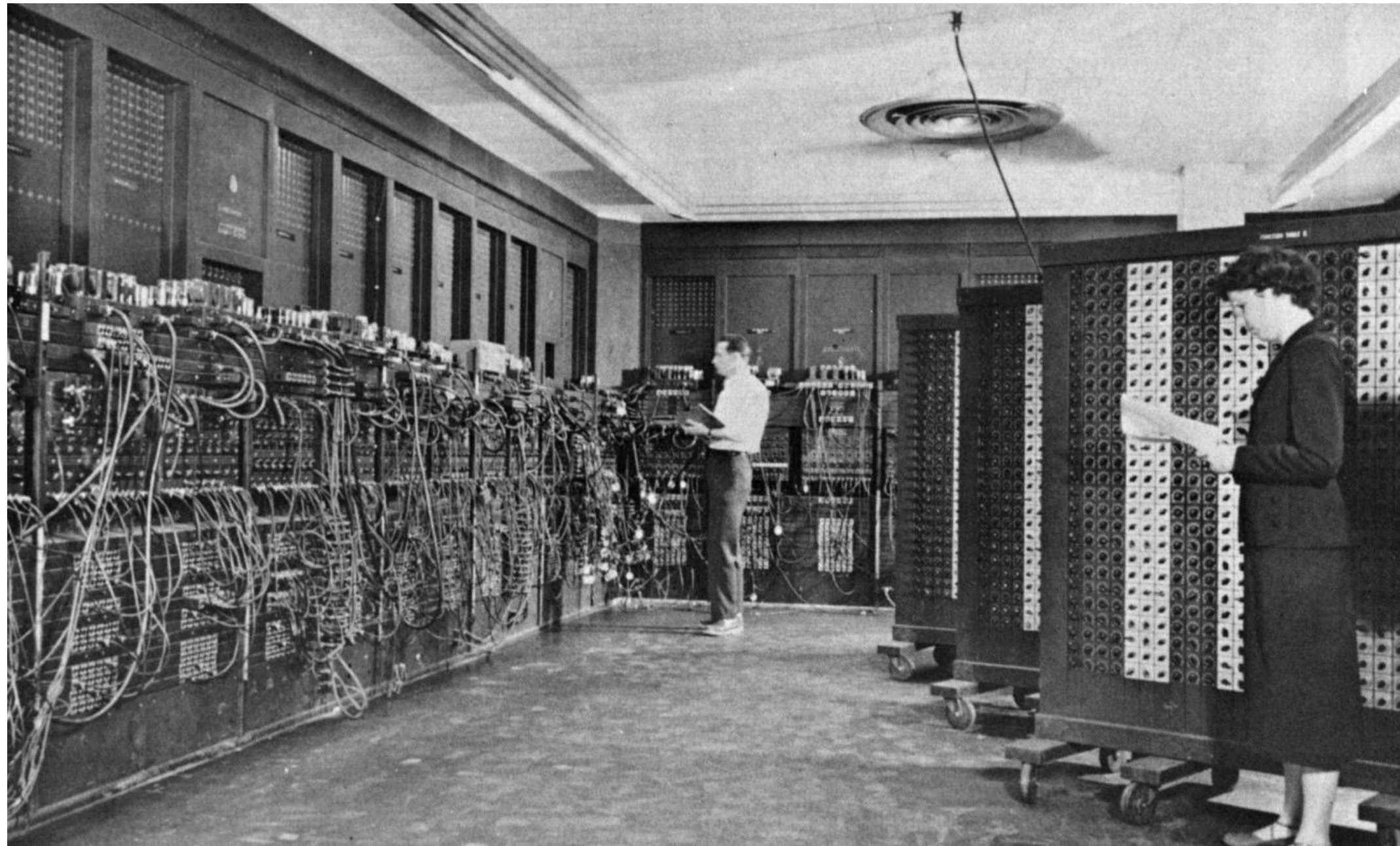
- 平时成绩 = 作业与测验*80% + 课堂表现与考勤*20%
- 期末成绩 = 期末实验成绩*60% + 期末理论成绩*40%
- 总评成绩 = 平时成绩*40% + 期末成绩*60%

□ 期末考试

- 形式：机试
- 理论：1月11日 (周一) 9:30-11:30
- 实验：1月11日 (周一) 14:30-17:30

本课程的目标

- 本课程主要目标是通过学习C语言，理解计算机是如何解决问题的并掌握程序设计思维



第一节：C语言概述

本节目标

- 1. 理解程序设计的基本概念，认识C语言及其主要特点**
- 2. 掌握第一个C程序的编写、编译与运行过程**
- 3. 认识C程序的基本结构和常见组成要素**

大模型明明能写代码，为什么我们还要学编程？

程序員處「食物鏈」底層 會被AI取代？

知·析趨勢
發布時間：2024/07/11

A+ A- 關注文章 儲存文章

分享： f 微博 邮件 链接



打开APP >

月入几万的程序员，要被AI取代了？

光锥智能 南财号
2025-02-13 14:51

+ 关注

BUSINESS

AI pushes 2026 tech layoffs past 50K in just three months, employers reveal

By Marc Vartabedian

Published April 2, 2026, 3:01 p.m. ET

Comments

25位IT大佬亲述：AI「吃掉」程序员！码农黄金时代终结

新智元 2025年05月03日 14:59 北京

刚刚，Claude最强模型Fable 5发布：性能爆炸，价格翻倍

Original 关注AI的 机器之心 2026年6月10日 07:44 北京



Spotify says its best developers haven't written a line of code since December, thanks to AI

AI确实已经很会编程了

❑ Claude Opus 4.6 可自主重新实现一个约 **1.69 万行代码**的生物信息学工具 gotree; 若由熟练工程师独立完成, 同类任务大约需 **2~17 周**

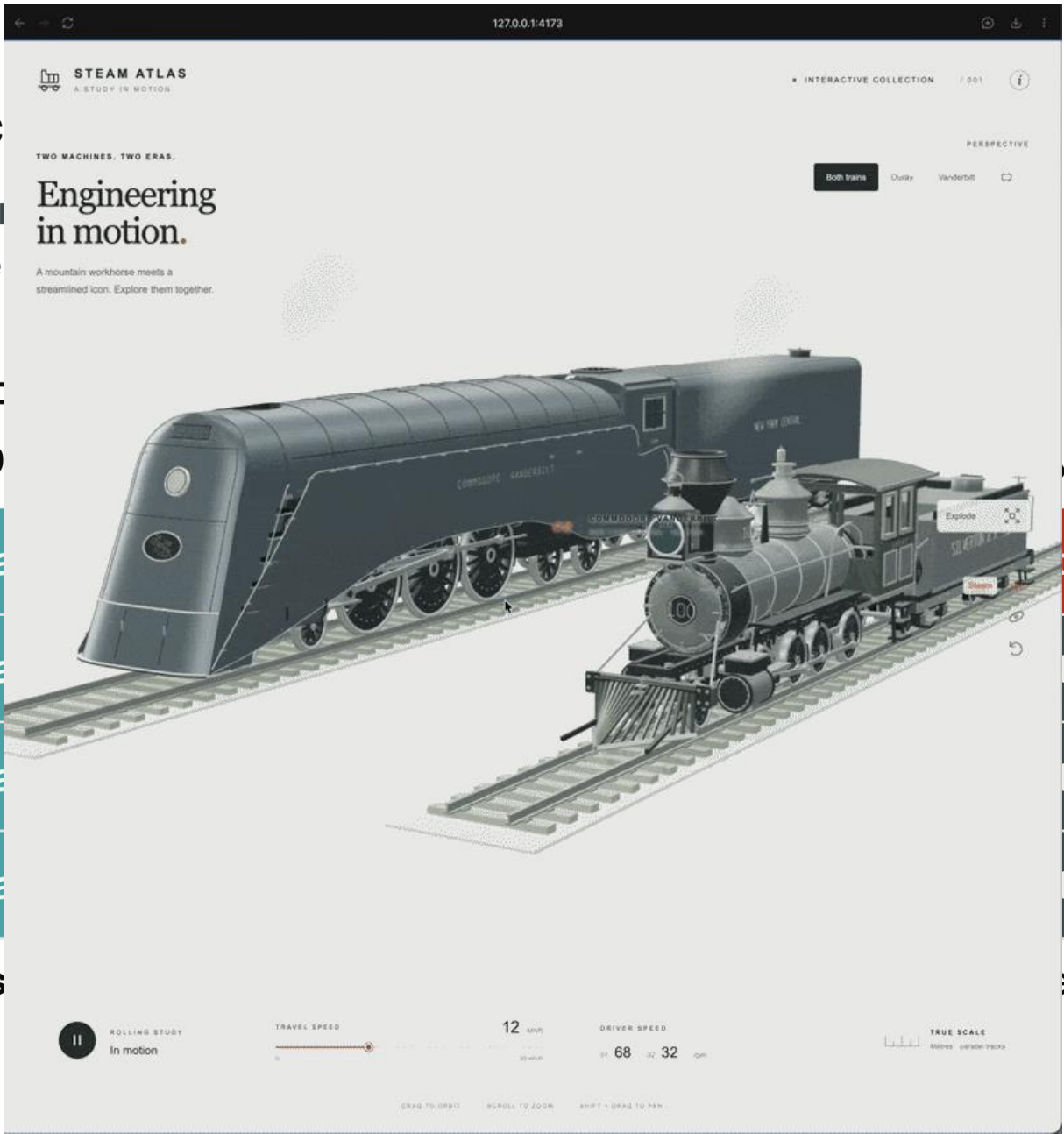
AI can rebuild c

In the MirrorCode be
provided a suite of te

cho
650

Opus 4.6	Pa
Opus 4.5	Pa
Opus 4.1	Pa
Opus 4.0	Pa

← Less



or, and is

Pkl

> 10,000 LoC

token limit*

t evaluated

t evaluated

t evaluated

e difficult →



Terminal-Bench Science 0.1
Scientific workflows

AutomationBench
Business workflows

FrontierMath Tier 4 (v2)
Research-level mathematics

Terminal-Bench 4.0
Computer terminal tasks

HealthBench Professional (length-adjusted)
Clinical tasks

BenchCAD
3D modeling

ARC-AGI-3
Novel interactive puzzles

OpenAI		Claude Fable 5.1
GPT-6 Astra	GPT-5.6 Sol	
• 64.6%	22.4%	52.6%
• 41.4%	18.1%	31.4%
• 97.6%	83.0%	87.8%
• 57.9%	37.3%	55.8%
• 63.4%	60.5%	58.1%
• 95.9%	83.3%	84.3%
• 99.9%	7.8%	—

Epoch AI, "MirrorCode: Evidence that AI can already do some weeks-long coding tasks", 2026-04-10.
<https://openai.com/index/gpt-6-astra/>

但AI可能会犯错，AI本身却无法担责.....

- ❑ AI可以生成代码、修改代码，但它仍可能出现逻辑错误、危险操作，造成严重的安全隐患
- ❑ AI越强，错误越隐蔽！

Meta用一整年证明Agent无法取代员工：事故增四成，工程师救火多七成

InfoQ Aug 28. 2026. 11:54 AM 北京

AI + ML

Cursor-Opus agent snuffs out startup's production database

Relax, the data's been recovered. Continue with your vibe coding

Thomas Claburn

48

Correcting the Financial Times report about recent Amazon.com service incidents and AI

AIR-2026-001 · AI AGENT INCIDENT REGISTER

Replit's coding agent deletes a production database during a code freeze

Incident: 2025-07-18 · Parties: Jason Lemkin / SaaStr (user); Replit, Inc. (platform operator)



但AI可能会犯错， AI本身却无法担责.....

你们现在大模型写的代码，还有精力 review 吗🤖

可不是么 现在大部分精力都在 review 代码

写方案 review 代码

就干这个了

小项目是甜品区 基本不用太操心

大颗粒项目你就等着吧

擦屁股擦到崩溃

ai 现在最不擅长的就是写代码

其他都干的不错

大模型上下文大的一般也就 1M

技术方案 各种 harness skill 加起来很容易就超了

上下文一丢他就开始自我发挥了

虽然各家现在都有上下文压缩，压缩次数多了，精度肯定就丢了



https://magic.solutionsuite.cn/r?t2... 11:13

的智能助理-dev 以下问题

必须修复：

1. GetLevelMin和getLevelItem 存在重复代码，代码不够复用。
2. resolveAdjustVersion直接从 seasonConfig解析version，不要扩散逻辑。
3. season的AdjustToLevel和 EnsureLevelNotDowngraded 重复了，保留一个即可。 EnsureLevelNotDowngraded 应该是season的内部函数， EnsureLevelNotDowngraded 是上层函数，应该依赖season的AdjustToLevel实现，现有实现层级定位混乱。
4. 不要改contract的TaskService 接口，本次的接口改动属于 task_svc 内部通用的 TaskService变更。



的智能助理-dev



智能助理-dev | 7*24小时冒烟干 11:48

已修复并推送。

改动点：

- contract.TaskService 已恢复为通用外部任务接口， TaskKeys 、 TaskView 、 IncrTask* 、聚合响应都收回到 task_svc 内部。
- task_svc.NewTaskService() 现在在内部聚合外部任务 ListTask 和内部任务进度 TaskViews ， IncrTask 也只使用 task_svc.IncrTaskRequest 。
- V2 团等级页删除独立 internal_task_loader ，统一由 task_loader 一次加载外部任务和内部任务视图。
- AdjustToLevel 改成 season 内部 adjustToLevel ，对外只保留 EnsureLevelNotDowngraded ； resolveAdjustVersion 只从 SeasonConfig.Version 解析； GetLevelMin 复用 getLevelItem 。

校验：

- git diff --check HEAD~1 HEAD 通过
- 相关包 go test -c -gcflags="all=-N -l" 编译通过
- coding_review 规则已拉取，Ponytail review 无需继续删除项
- 本地轻量增量代码分：100/100（阈值

沟通时请保持“公开可接受”



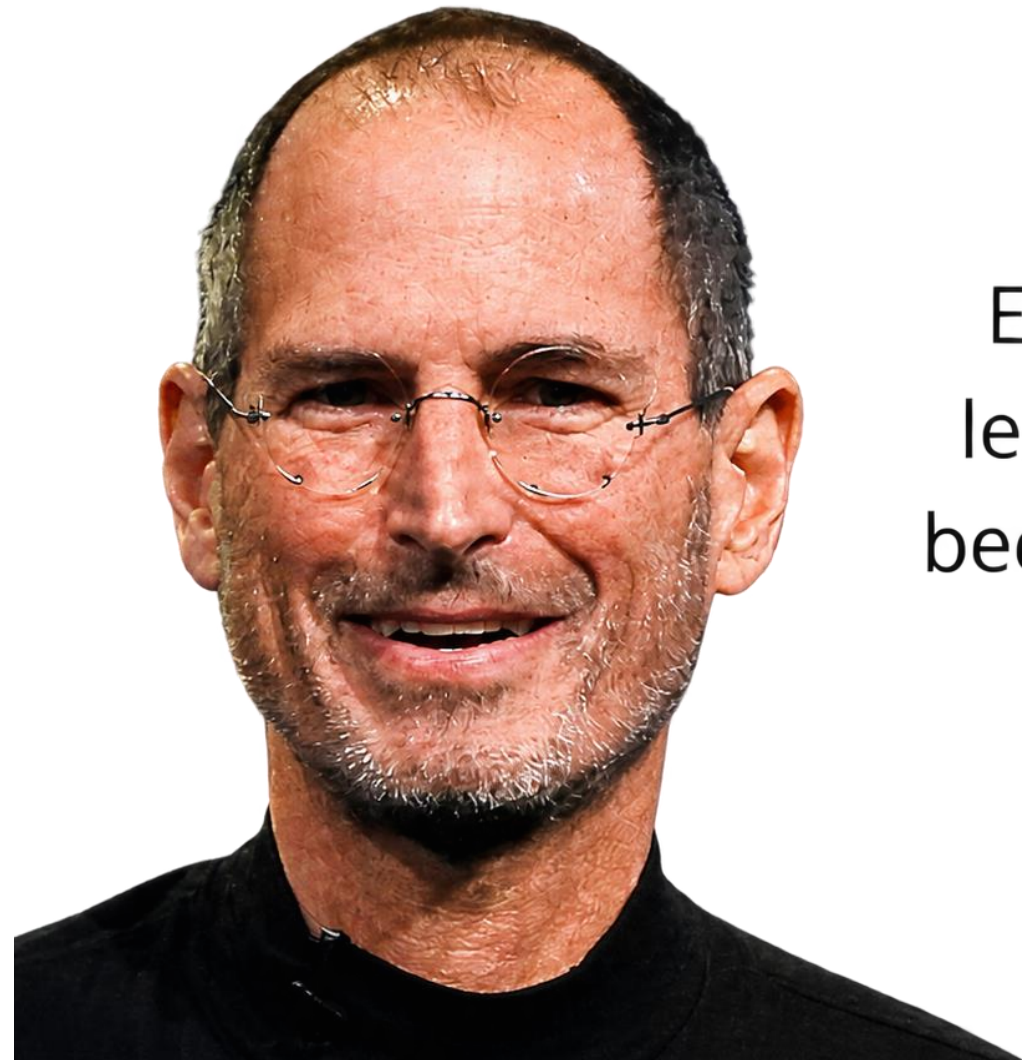
掌握编程，才能更好的驾驭AI

人亲手写每行代码



人定义问题 → AI辅助实现 → 人审查并负责

- AI能显著提高编程效率，同时也对计算机专业学生的编程水平提出了更高的要求

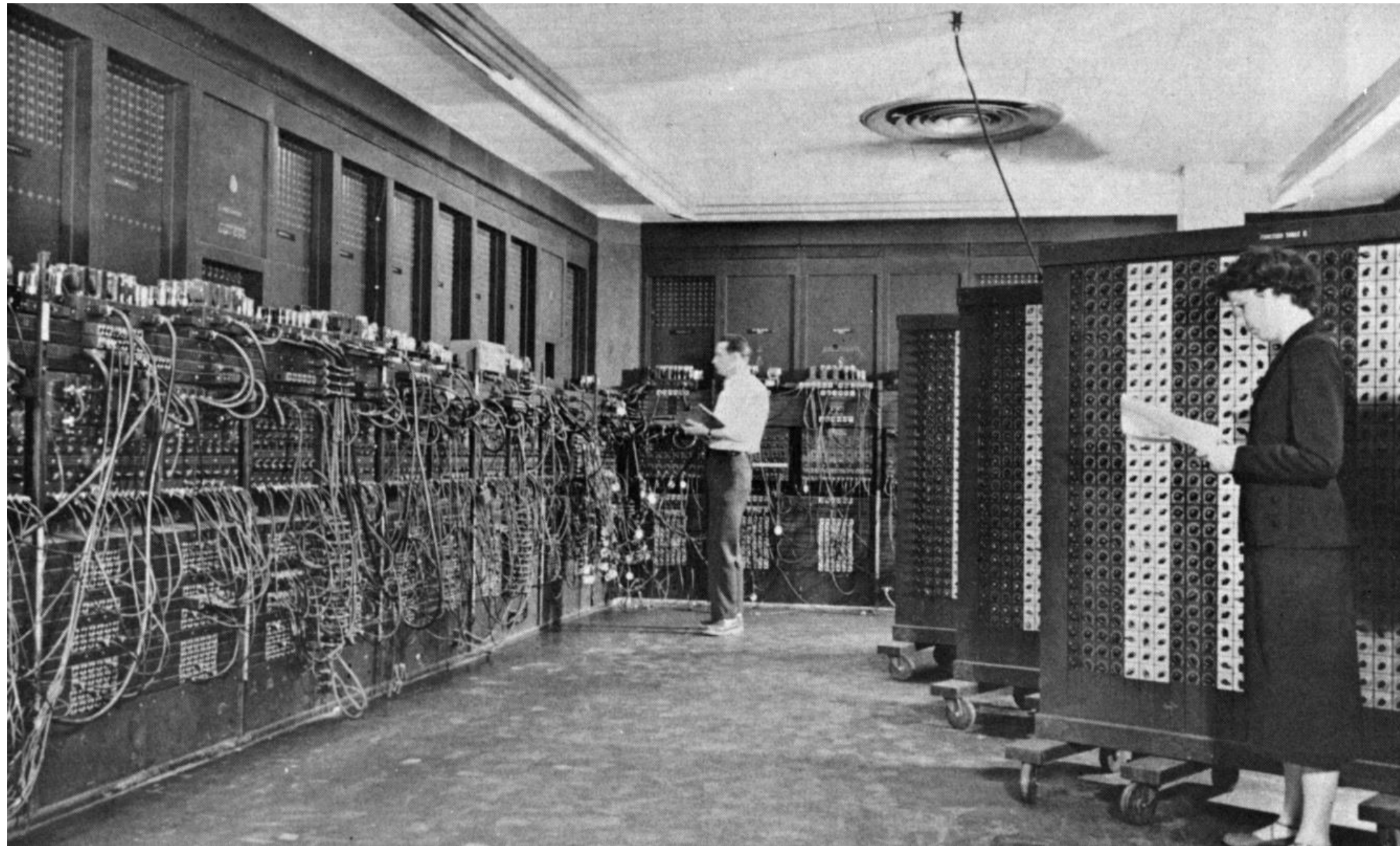


Everybody in this country should
learn how to program a computer,
because it teaches you how to think.

— Steve Jobs —

本课程的目标

- 本课程主要目标是通过学习C语言，理解计算机是如何解决问题的并掌握程序设计思维



✓ 能设计!

✓ 能评估!

学习过程中如何使用大模型？

目的
能独立编写C程序

VS

手段
利用AI提高学习质量

三条使用原则

1. 想清楚，说清楚

在使用AI时，要提好问题，能清楚表达需求

2. 多问“为什么”

让 AI 解释概念、逐行分析代码、找不足、举反例，帮助自己理解透彻

3. 代码必须独立完成 !!

代码要自己独立写，不要拄着拐杖练习走路

提示词

✓ 推荐

- 这个报错是什么意思？
- 这个算法背后的原理是什么？
- 我代码有什么可以优化的空间吗？
- 帮我设计几个测试用例？

✗ 不推荐

- “这道题目的C代码是什么？”
- “测试没通过，帮我改一下代码。”
- “不用解释，只给答案。”

什么是程序？

□ 计算机只能执行**清晰具体的指令**

□ “两数相加”

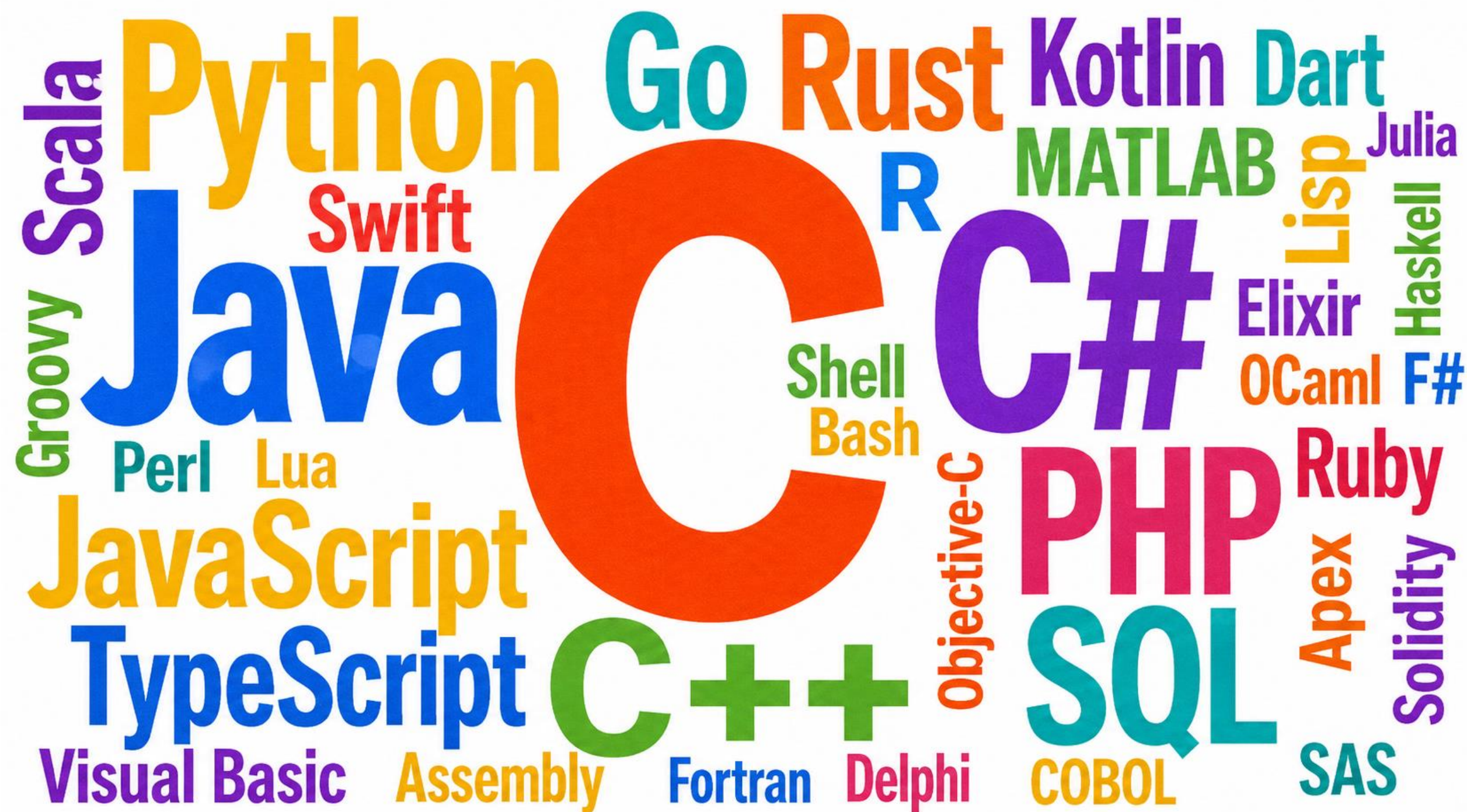
1. 从**内存位置2000**把一个数字拷贝到**寄存器1**
2. 从**内存位置2004**把另一个数字拷贝到**寄存器2**
3. 把**寄存器2**中内容与**寄存器1**中的内容相加，结果存储在**寄存器1**中
4. 把**寄存器1**中的内容拷贝到**内存位置2008**

□ **指令**：可以被计算机理解并执行的基本操作命令

□ **程序**：计算机能识别和执行的、用来完成特定功能的一组指令

编程语言：计算机能听懂的语言

□ 编程语言：编写计算机程序的标准化语言



机器语言：最“直接的”编程语言

- 在计算机的内部，主要的电子元件都是由晶体管构成的，这就意味着计算机只能理解 **0 (低电平)** 和 **1 (高电平)**
- 计算机能直接识别的二进制代码成为**机器指令**，机器指令的集合就是**机器语言**
- 两数相加

1. 从内存位置2000把一个数字拷贝到寄存器1
2. 从内存位置2004把另一数字拷贝到寄存器2
3. 把寄存器2中内容与寄存器1中的内容相加，结果存储在寄存器1中
4. 把寄存器1中的内容拷贝到内存位置2008

00000001 00000001 00000111 11010000
00000001 00000010 00000111 11010100
00000011 00000001 00000010
00000010 00000001 00000111 11011000

机器语言实现

机器语言：最“直接的”编程语言

□ 计算32和26最大公约数：

1. 把 u 设为 32
2. 把 v 设为 26
3. 当 v 不是 0 的时候，重复下面的步骤：
 1. 计算 u 除以 v 的余数，记作 temp
 2. 把 u 的值改成 v
 3. 把 v 的值改成 temp
1. 循环结束后，u 就是最大公约数
2. 输出 u

CF	FA	ED	FE	07	00	00	01	03	00	00	00	01	00	00	00	03	00	00	00	F0	01	00	00	00	20	00	00	00	00	00	00	00	19	00	00	00	88	01	00	
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	D0	00	00	00	00	00	00	00	00	10	02	00	00	00	00	
00	00	D0	00	00	00	00	00	00	00	07	00	00	00	07	00	00	00	04	00	00	00	00	00	00	00	5F	5F	74	65	78	74	00	00	00	00	00	00	00		
00	00	00	5F	5F	54	45	58	54	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	66	00	00	00	00	00	00	10	02	00	00	00		
04	00	00	00	E0	02	00	00	02	00	00	00	00	04	00	80	00	00	00	00	00	00	00	00	00	00	00	00	5F	5F	63	73	74	72	69	6E	67	00	00		
00	00	00	00	00	5F	5F	54	45	58	54	00	00	00	00	00	00	00	00	00	00	66	00	00	00	00	00	00	00	00	03	00	00	00	00	00	00	76	02	00	
00	00	00	00	00	00	00	00	00	00	00	00	00	00	02	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	5F	5F	63	6F	6D	70	61	63	74		
5F	75	6E	77	69	6E	64	5F	5F	4C	44	00	00	00	00	00	00	00	00	00	00	00	00	70	00	00	00	00	00	00	00	20	00	00	00	00	00	00	00	00	
80	02	00	00	03	00	00	00	F0	02	00	00	01	00	00	00	00	00	00	02	00	00	00	00	00	00	00	00	00	00	00	00	5F	5F	65	68	5F	66	72		
61	6D	65	00	00	00	00	00	00	5F	5F	54	45	58	54	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	40	00	00	00	00	00	00	
00	00	A0	02	00	00	03	00	00	00	00	00	00	00	00	00	00	00	0B	00	00	68	00	00	00	00	00	00	00	00	00	00	00	00	02	00	00	00	18	00	
00	00	00	F8	02	00	00	05	00	00	00	48	03	00	00	2C	00	00	00	0B	00	00	00	50	00	00	00	00	00	00	00	02	00	00	00	02	00	00	00	00	
02	00	00	00	04	00	00	00	01	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	55	48	89	E5	48	83	EC	20	C7	45	FC	00	00	00	00	C7	45	F8	00	
20	00	00	00	C7	45	F4	1A	00	00	00	81	7D	F4	00	00	00	00	0F	84	1B	00	00	00	8B	45	F8	99	F7	7D	F4	89	55	F0	8B	45	F4	89	45	00	
F8	8B	45	F0	89	45	F4	E9	D8	FF	FF	FF	48	8D	3D	00	00	00	00	8B	75	F8	B0	00	E8	00	00	00	00	BE	00	00	00	00	89	45	EC	89	F0	00	
48	83	C4	20	5D	C3	25	64	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	66	00	00	00	00	00	00	01	00	00	00	00	00	00	00	00	00
00	00	00	00	00	00	00	00	00	14	00	00	00	00	00	00	00	00	01	7A	52	00	01	78	10	01	10	0C	07	08	90	01	00	00	24	00	00	00	1C	00	00
00	00	50	FF	FF	FF	FF	FF	FF	FF	66	00	00	00	00	00	00	00	00	41	0E	10	86	02	43	0D	06	00	00	00	00	00	00	00	52	00	00	00	04	00	00
00	00	2D	48	00	00	00	00	00	00	1D	00	00	00	00	01	00	00	06	18	00	00	00	0E	02	00	00	66	00	00	00	00	00	00	00	1F	00	00	00	00	00
0E	04	00	00	90	00	00	00	00	00	00	00	01	00	00	00	0F	01	00	00	00	00	00	00	00	00	00	00	00	00	00	00	0F	04	00	00	A8	00	00		
00	00	00	00	00	07	00	00	00	01	00	00	00	00	00	00	00	00	00	00	00	00	5F	6D	61	69	6E	00	5F	70	72	69	6E	74	66	00	5F	6D	61	00	00
69	6E	2E	65	68	00	4C	5F	2E	73	74	72	00	45	48	5F	66	72	61	6D	65	30	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

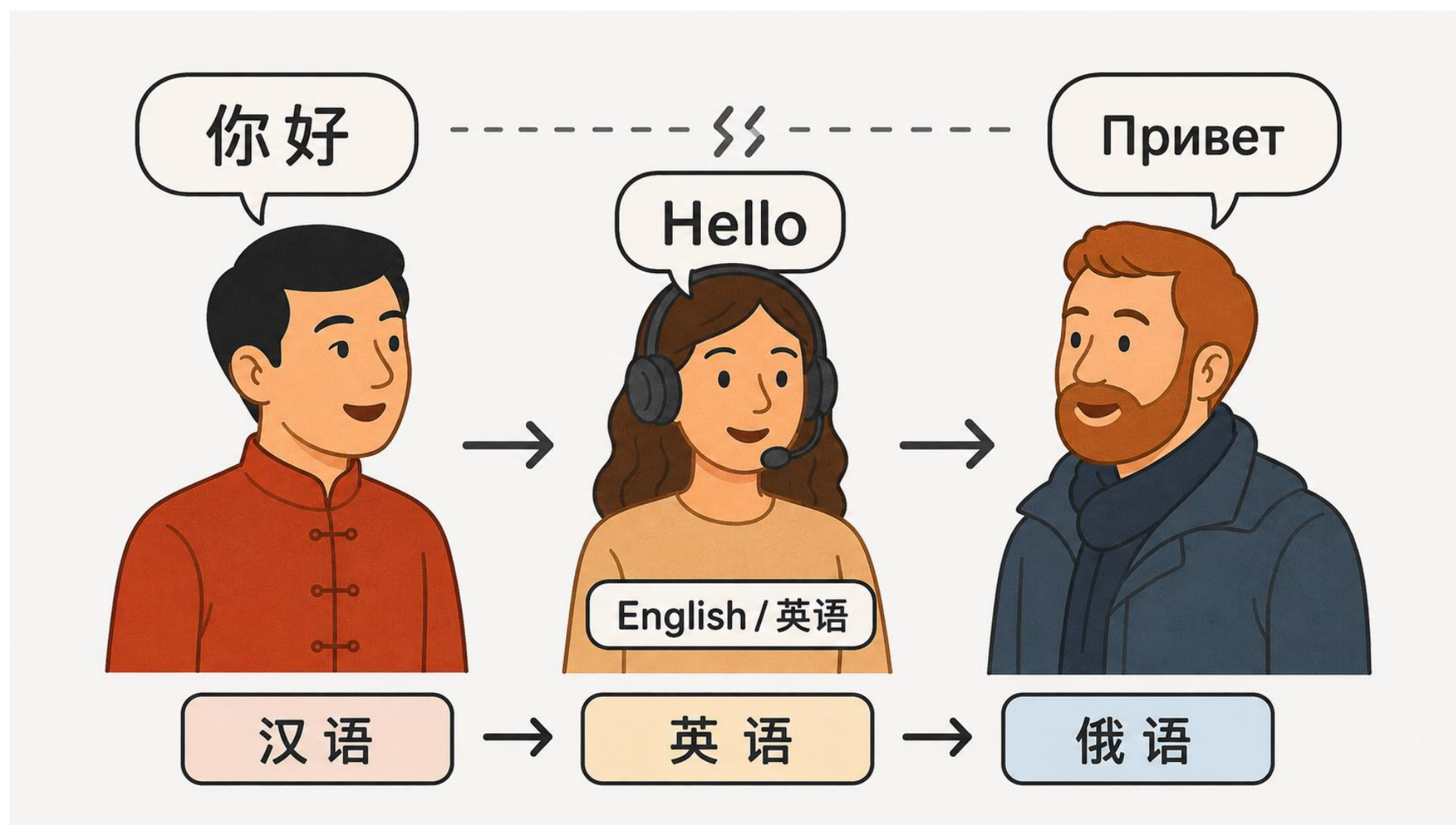
机器语言：最“直接的”编程语言



雅卡尔织布机 (Jacquard machine)

机器语言有什么问题？

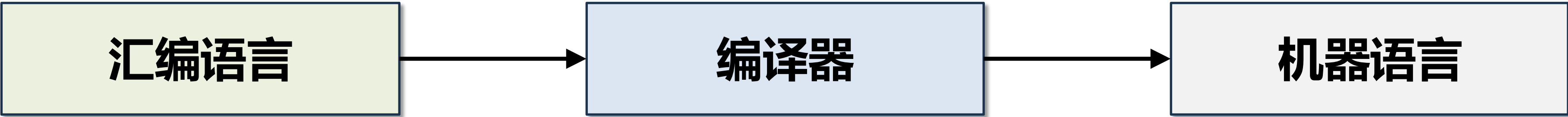
- ❑ 机器语言**繁冗又难理解**，难道只能靠机器语言来和计算机打交道吗？
- ❑ 想象一下，假设你完全不会俄语，但你需要与俄国人打交道，怎么办？
- ❑ 答案是**借助一种中间语言**，比如英语，实现汉语 → 英语 → 俄语的交流



汇编语言：对机器语言的“翻译”

□ 借助汇编语言，我们可以将一些常见的机器指令转化成人类可以理解的**助记符**，比如规定：

汇编指令	机器指令	汇编指令	机器指令
ADD EAX, EBX 加法	01 D8	SUB EAX, EBX 减法	29 D8
IMUL EAX, EBX 乘法	0F AF C3	IDIV EBX 除法	F7 FB
MOV EAX, EBX 数据传送	89 D8	CMP EAX, EBX 比较	39 D8
AND EAX, EBX 逻辑与	21 D8	OR EAX, EBX 逻辑或	09 D8
INC EAX 加一	FF C0	DEC EAX 减一	FF C8



汇编语言实现 “两数相加”

□ 两数相加

1. 从内存位置2000把一个数字拷贝到寄存器1
2. 从内存位置2004把另一数字拷贝到寄存器2
3. 把寄存器2中内容与寄存器1中的内容相加，结果存储在寄存器1中
4. 把寄存器1中的内容拷贝到内存位置2008

01 01 07 D0
01 02 07 D4
03 01 02
02 01 07 D8

机器语言实现

```
mov eax, dword [0x07D0]
mov ebx, dword [0x07D4]
add eax, ebx
mov dword [0x07D8], eax
```

汇编语言实现 (x86)

汇编语言有什么问题？

- 汇编语言与底层的处理器架构绑定，换机器就需要重写，使得程序移植性差
- 能否可以用更“直观”的方式表达我们的想法？
- 如“两数相加”直接表示为类似数学公式： $\text{sum} = a + b$ ？

本门课的主角：C语言

- C语言则是对汇编语言的进一步抽象，不受限于特定的 CPU 和指令集，使得人类更容易理解

```
01 01 07 D0  
01 02 07 D4  
03 01 02  
02 01 07 D8
```

机器语言实现

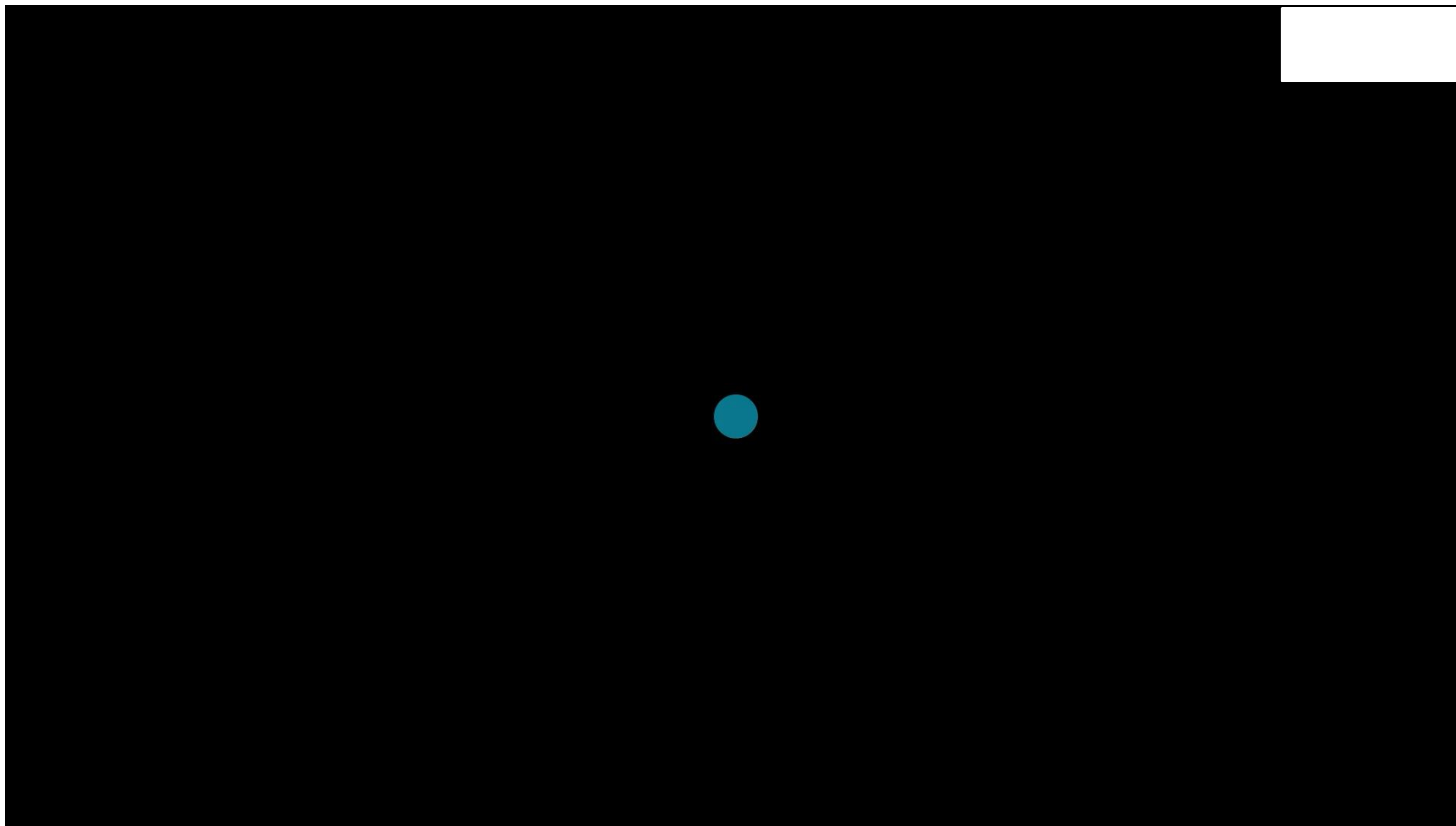
```
mov eax, dword [0x07D0]  
mov ebx, dword [0x07D4]  
add eax, ebx  
mov dword [0x07D8], eax
```

汇编语言实现 (x86)

```
sum = a + b;
```

C语言实现

C语言简史



https://www.bilibili.com/video/BV1Eb411b716/?spm_id_from=333.337.search-card.all.click&vd_source=ba8bb8c9887aff7403e9af7acc7e5ad9

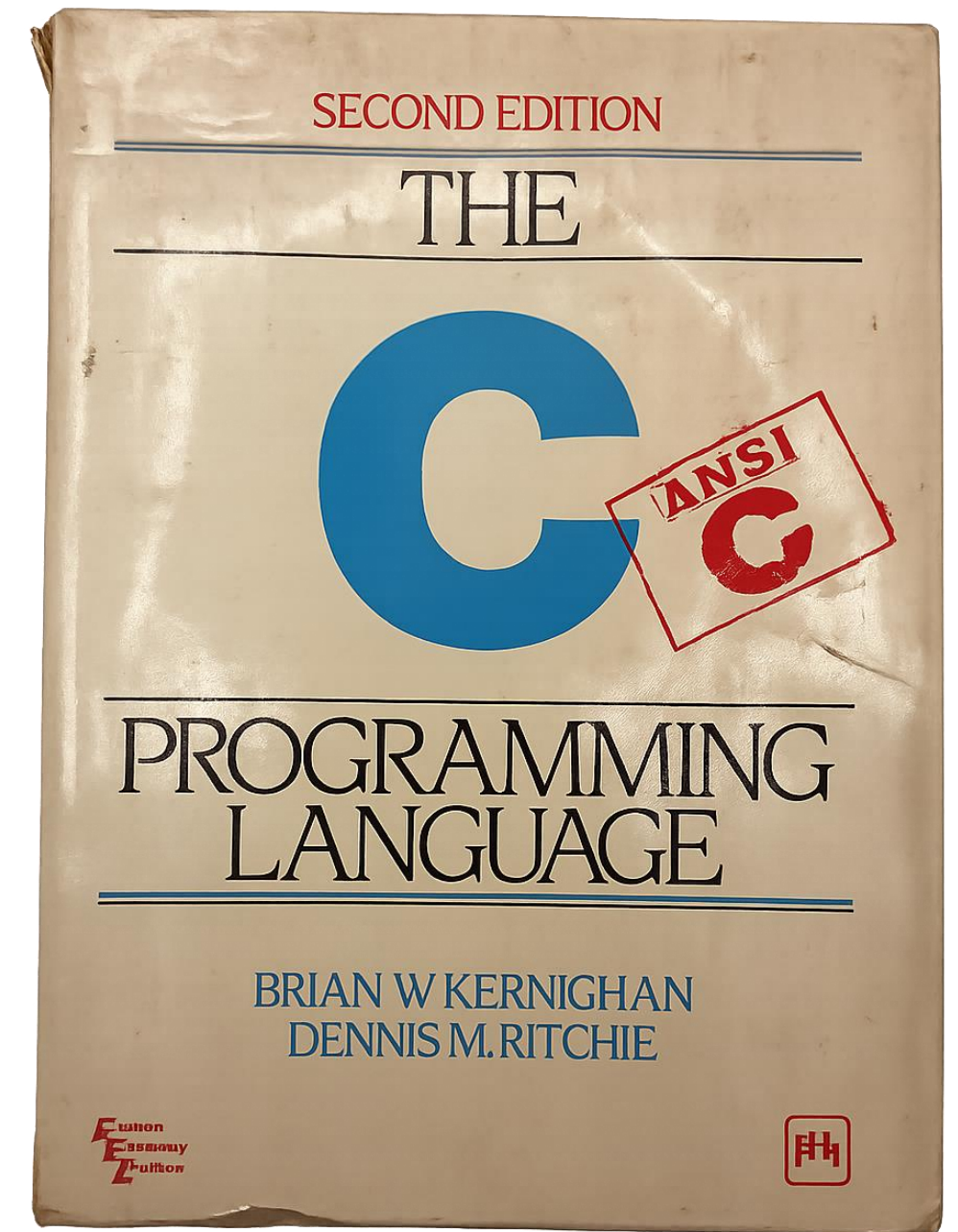
C语言简史



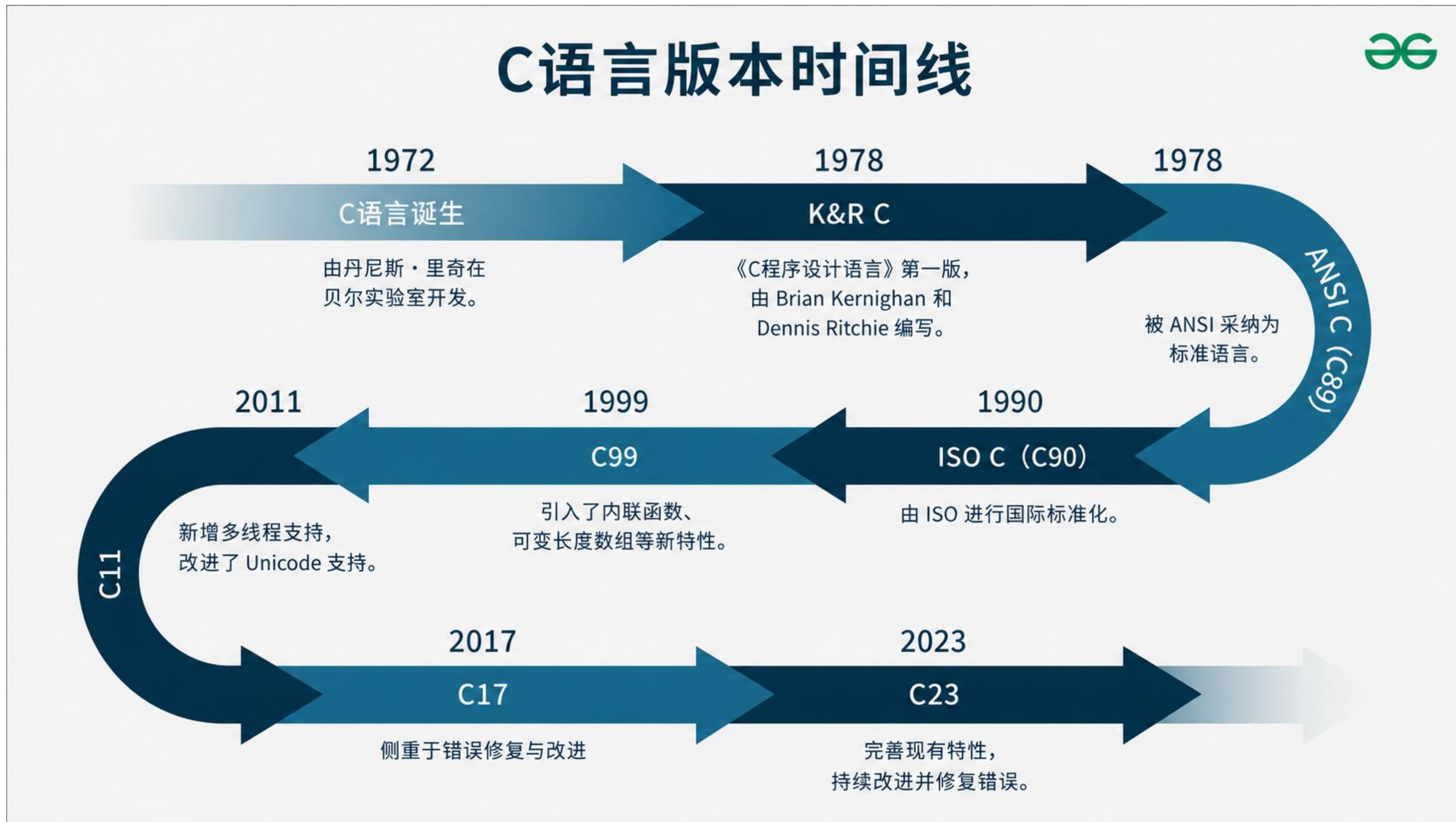
肯·汤普森 (Ken Thompson) 和丹尼斯·里奇 (Dennis Ritchie)

C语言标准

- ❑ 最初C语言只能在大型计算机上执行，移植到微机上出现了许多**不同的版本**
- ❑ 1978年，布莱恩·柯林汉和丹尼斯·里奇合著的《**C Programming Language**》(K&R) 是第一版公认C标准：
- ❑ 1983年，美国国家标准化协会（ANSI）为制定了第一个正式C标准**ANSI C**，定义了**C标准**和**C标准库**

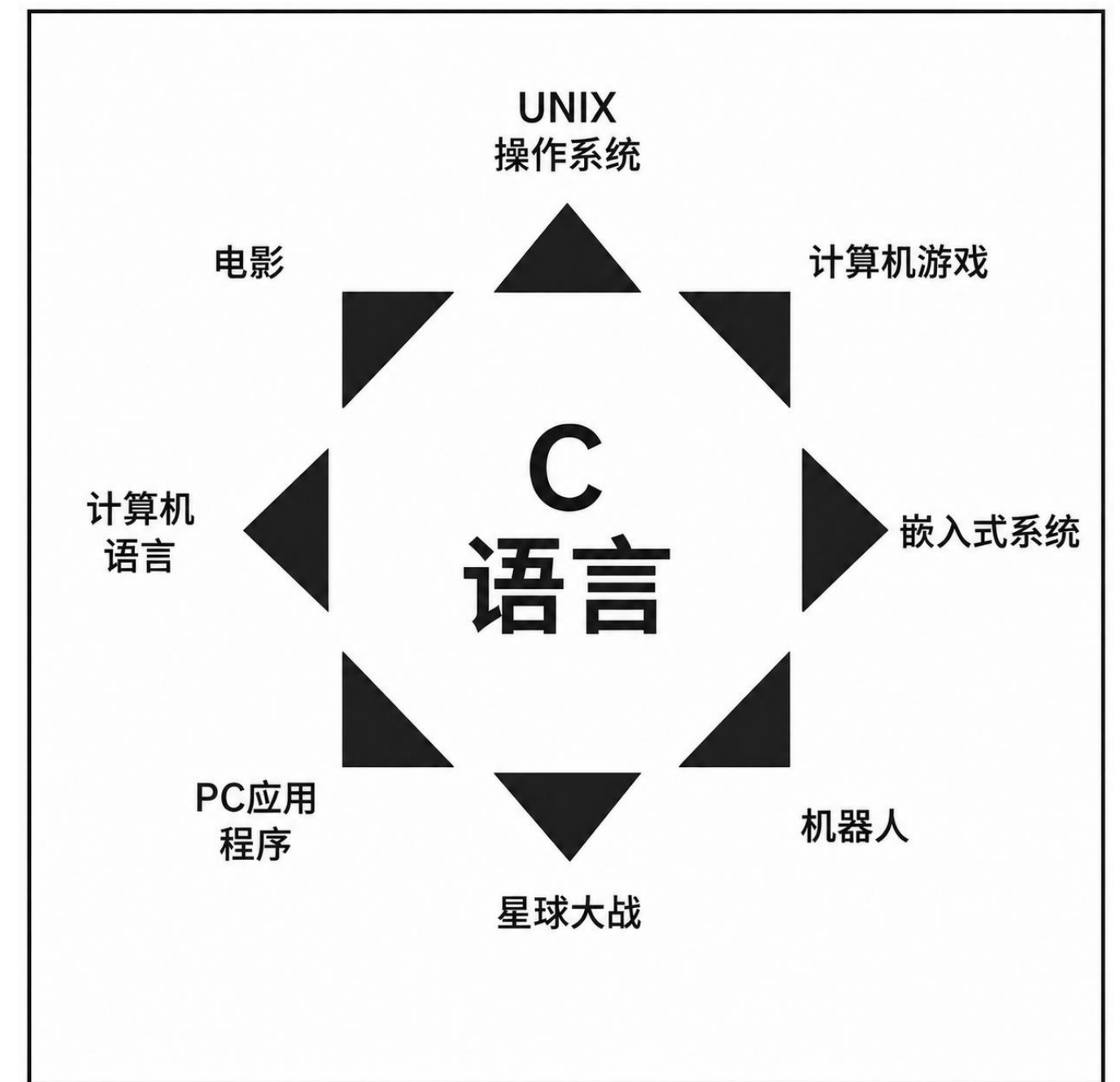
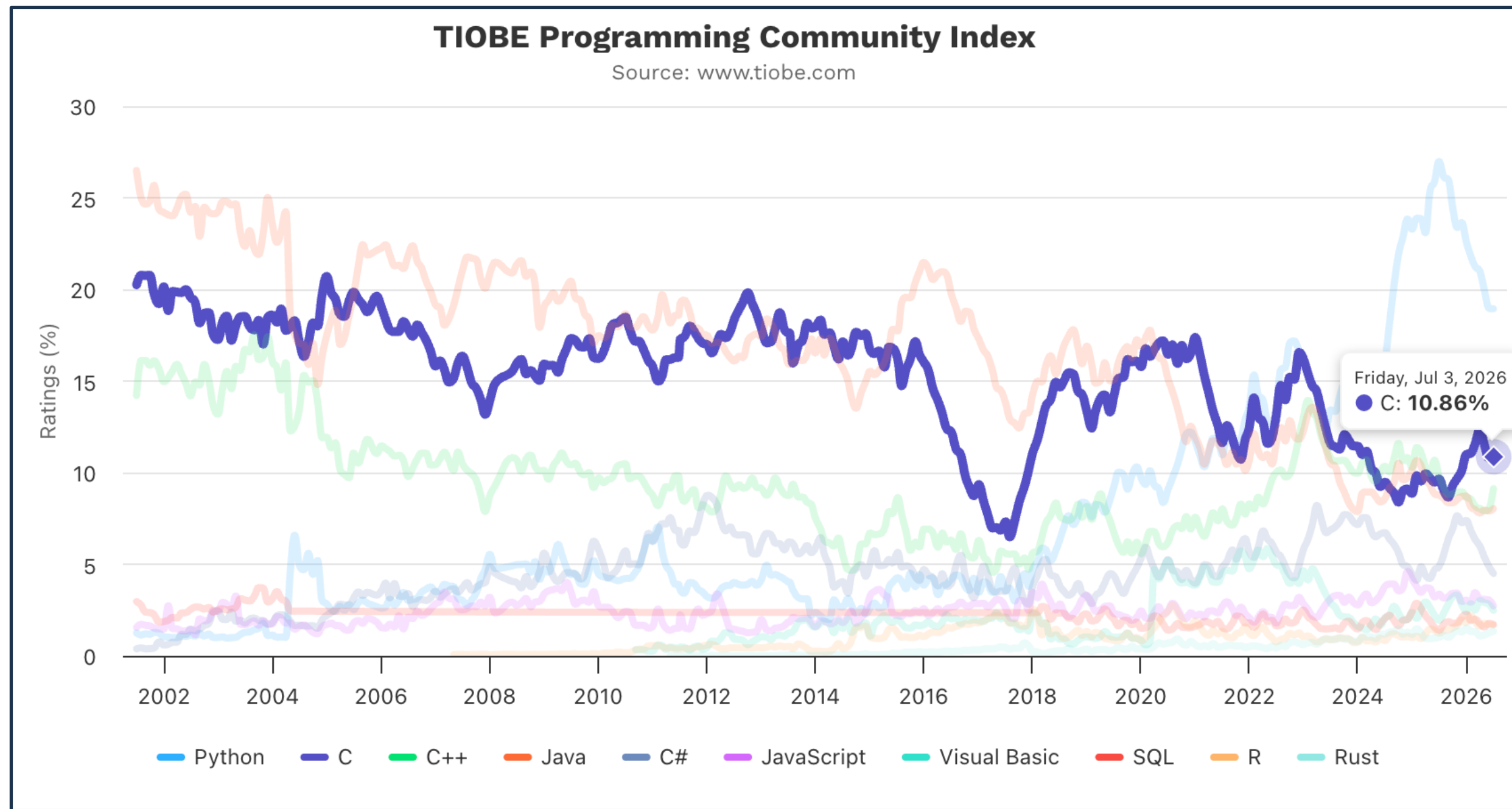


C语言标准



为什么学C?

□ 过去40年最重要、最流行的编程语言之一



<https://www.tiobe.com/tiobe-index/>

为什么学C?

□ C语言的优势

- **用户友好**：易读、易写
- **简洁**：语法精简、编译系统简洁
- **高效**：具备 CPU、内存微调控制能力
- **灵活**：不同操作系统间可移植性强

□ 现代编程语言语法差异很小，几乎都是 **C-like** 语言

```
#include <stdio.h>
int main(void)
{
    printf("Hello, world\n");
    return 0;
}
```

C

```
class Hello {
    public static void main(String[] arg) {
        System.out.println("Hello, world");
    }
}
```

Java

```
print("Hello, world\n")
```

Python

C语言编程环境设置：VS Code

- ❑ **集成开发环境** (IDE, Integrated Development Environment) 是用于提供程序开发环境的应用程序，一般包括代码编辑器、编译器、调试器和图形用户界面等工具
- ❑ 本课程推荐使用 **Visual Studio Code (VS Code)** 或 DEV C++

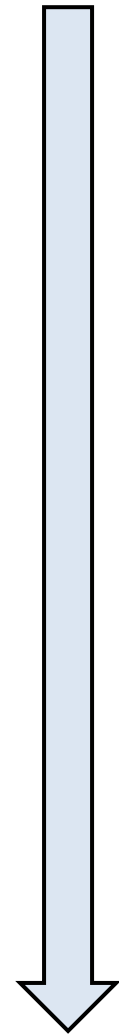


第一个C语言程序

```
/* My first C source file */  
#include <stdio.h>  
  
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

示例解释

- helloworld.c 代码虽短，但五脏俱全，我们按照程序从上到下的顺序解析一下该代码



```
/* My first C source file */  
#include <stdio.h>  
  
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

C语言程序的基本结构：注释

```
/* My first C source file */
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    printf("Hello, world!\n");
```

```
    return 0;
```

```
}
```

块注释

```
/* 一条注释 */
```

```
/* 一条注释  
   第二条注释 */
```

单行注释

```
// 一条注释
```

```
// 一条注释  
// 第二条注释
```

- 首先，**/**/** 括起来的部分是**注释**
- 注释在代码中**不参与程序的执行**，仅是为了方便阅读代码而做的注解
- C语言常用的注释有两种形式：**块注释**和**单行注释**

C语言程序的基本结构：预处理与头文件

```
/* My first C source file */  
#include <stdio.h> //预处理指令  
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

❑ **#include<>** 尖括号括号中部分是**头文件**

- 不同头文件提供不同的库函数及相关信息
- **stdio.h** (标准输入输出头文件) 声明了下面将会用到的 **printf()**

❑ **#include** 是预处理指令：将指定头文件的内容包含到当前源程序中

- 在正式编译之前，预处理阶段会先处理这类指令

C语言程序的基本结构：主函数

```
/* My first C source file */  
#include <stdio.h>  
int main(void) { //定义主函数  
    printf("Hello, world!\n");  
    return 0;  
}
```

- ❑ **main** 函数是C程序的**入口**，程序从 main 函数开始执行
- ❑ **int** 是函数的**返回类型**，表示 main 函数执行结束后，可以通过 **return** 向操作系统返回一个整数
- ❑ **void** 表示 main 函数**不接收参数**
- ❑ **花括号**括起的部分称为**函数体**，其中包含 main 函数要执行的语句

C语言程序的基本结构：语句与函数

```
/* My first C source file */
```

```
#include <stdio.h>
```

```
int main(void) {
```

```
    printf("Hello, world!\n");
```

```
    return 0;
```

```
}
```



；表示这条语句结束

- ❑ printf("Hello, world!\n"); 是一条**语句**，语句是程序执行的基本单位
- ❑ printf 是C标准库提供的**输出函数**，其函数声明位于 stdio.h 头文件中
 - 函数用于在屏幕上输出信息
 - 这里打印的是：Hello, world!
 - \n 是换行符，输出 Hello, world! 后会换一行

C语言程序的基本结构：return语句

```
/* My first C source file */  
#include <stdio.h>  
int main(void) {  
    printf("Hello, world!\n");  
    return 0; //返回语句  
}
```

- ❑ **return** 是返回语句，执行到 return 时，当前函数会**结束执行**
- ❑ return 将**返回**紧随在它后面的值 **0**
- ❑ 返回给谁呢？返回给了操作系统，表示**程序正常结束**
- ❑ 返回非 0 值通常用于表示程序出现异常或未能正常完成

C语言程序的基本结构：return语句

```
/* My first C source file */  
#include <stdio.h>  
int main(void) {  
    printf("Hello, world!\n");  
    return 0;  
}
```

- ❑ main 函数的两行代码都具有分号；
- ❑ 分号在C语言中实际上与汉语的句号是一致的，它表明了一个代码语句的结束
- ❑ **注意是半角分号 “;” 而不是全角分号 “；”**

简单C程序的基本结构

- 一个简单的C程序通常包含**预处理指令**、**函数**和函数中的**语句**

预处理指令



```
#include <stdio.h>
```

函数



```
int main(void) {
```

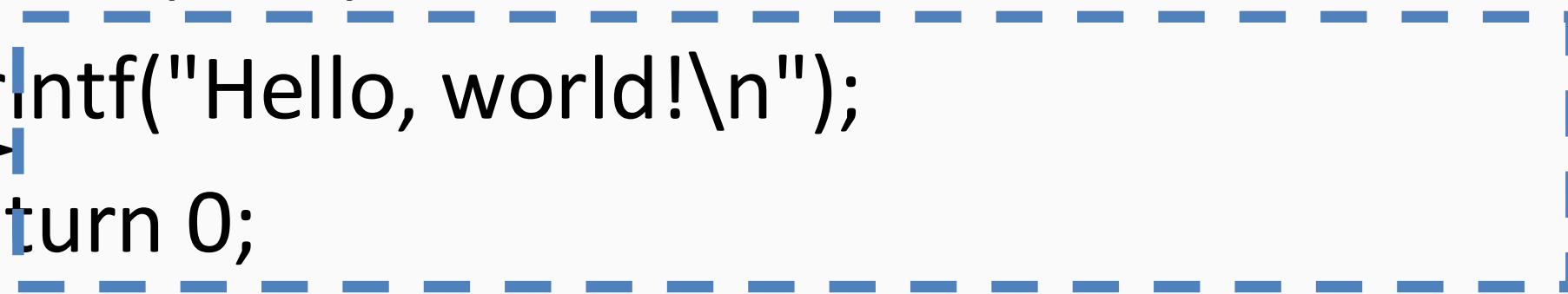
语句



```
printf("Hello, world!\n");
```

```
return 0;
```

```
}
```



练习

□ 尝试修改程序看下是否能打印出以下图形？

```
*  
***  
*****
```

示例

□ 猜一猜：以下程序会输出什么？

```
#include <stdio.h>

int main(void) {
    int a, b, sum;
    a = 123;
    b = 456;
    sum = a + b;
    printf("sum is %d\n", sum);

    return 0;
}
```

示例：两数相加

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int a, b, sum;
```

```
    a = 123;
```

```
    b = 456;
```

```
    sum = a + b;
```

```
    printf("sum is %d\n", sum);
```

```
    return 0;
```

```
}
```

定义三个整型变量 a、b、sum

示例：两数相加

```
#include <stdio.h>

int main(void) {
    int a, b, sum;
    a = 123;
    b = 456;
    sum = a + b;
    printf("sum is %d\n", sum);

    return 0;
}
```

对变量 a,b 赋值

"=" 是赋值运算符

示例：两数相加

```
#include <stdio.h>

int main(void) {
    int a, b, sum;
    a = 123;
    b = 456;
    sum = a + b;
    printf("sum is %d\n", sum);

    return 0;
}
```

计算 $a+b$ 的值，并将结果赋值给 `sum`

示例：两数相加

```
#include <stdio.h>

int main(void) {
    int a, b, sum;
    a = 123;
    b = 456;
    sum = a + b;
    printf("sum is %d\n", sum);

    return 0;
}
```

输出： sum is 579

%d表示在这里输出一个**十进制整数**
输出时用 sum 的值取代 %d

printf("sum is %d\n", sum);

```
#include<stdio.h>
```

```
int main(void){  
int a=3,b=45,c=2,d=8,e=12,f;  
f=a*b+c*d+e;  
printf("Total = %d yuan\n",f);  
return 0;  
}
```

```
#include <stdio.h>
```

```
int main(void)  
{  
    /* 商品数量和单价 */  
    int book_count = 3;  
    int book_price = 45;  
    int pen_count = 2;  
    int pen_price = 8;  
    int shipping_fee = 12;  
    int cost;  
  
    /* 计算订单总金额 */  
    cost = book_count * book_price  
        + pen_count * pen_price  
        + shipping_fee;  
  
    printf("Total = %d yuan\n", cost);  
  
    return 0;  
}
```

程序可读性

- 尽量选择有意义的函数名与变量标识符
- 适当添加注释
- 适当添加空格、空行

```
#include <stdio.h>

int main(void)
{
    /* 商品数量和单价 */
    int book_count = 3;
    int book_price = 45;
    int pen_count = 2;
    int pen_price = 8;
    int shipping_fee = 12;
    int cost;

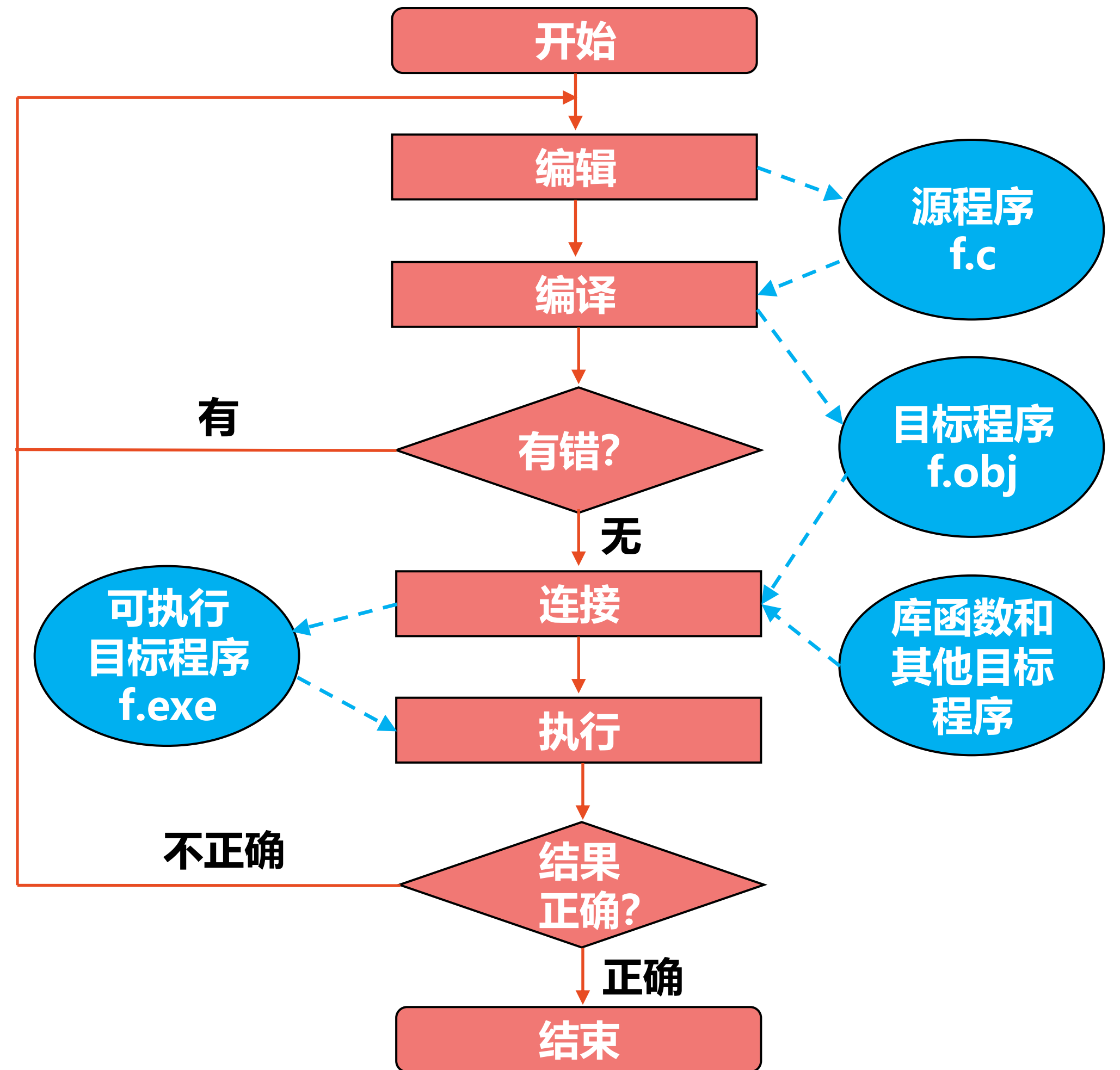
    /* 计算订单总金额 */
    cost = book_count * book_price
          + pen_count * pen_price
          + shipping_fee;

    printf("Total = %d yuan\n", cost);

    return 0;
}
```

编译过程

- 1. 编辑：**编写和修改C源程序，并保存为 .c 文件
- 2. 编译：**检查程序并将源程序翻译成 .obj 目标文件
- 3. 链接：**将目标文件与所需的库函数等组合，生成可执行 .exe 程序
- 4. 执行：**运行可执行程序，观察程序的运行结果
- 5. 检查：**如果出现错误或结果不正确，需要修改源程序并重新编译



调试程序

□ 以下程序有什么问题？编译试试看？

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!")
    return 0;
}
```

总结

1. C语言是一种通用、高效、可移植性较好的高级编程语言
2. 简单C程序通常包含预处理指令、函数和语句；main 是C程序的入口函数
3. 变量用于保存数据，表达式和赋值用于完成计算
4. 函数 printf() 用于格式化输出，return 0; 通常表示程序正常结束
5. 良好的变量命名、适当的注释和规范的排版可以提高程序的可读性
6. C程序通常经历：编辑 → 编译 → 链接 → 执行

作业

- 阅读《C程序设计》第1章 (P9-P11了解即可)
- 完成 PTA 本周习题

补充

计算机的组成

□ 计算机主要由CPU、内存、硬盘、输入/输出装置组成

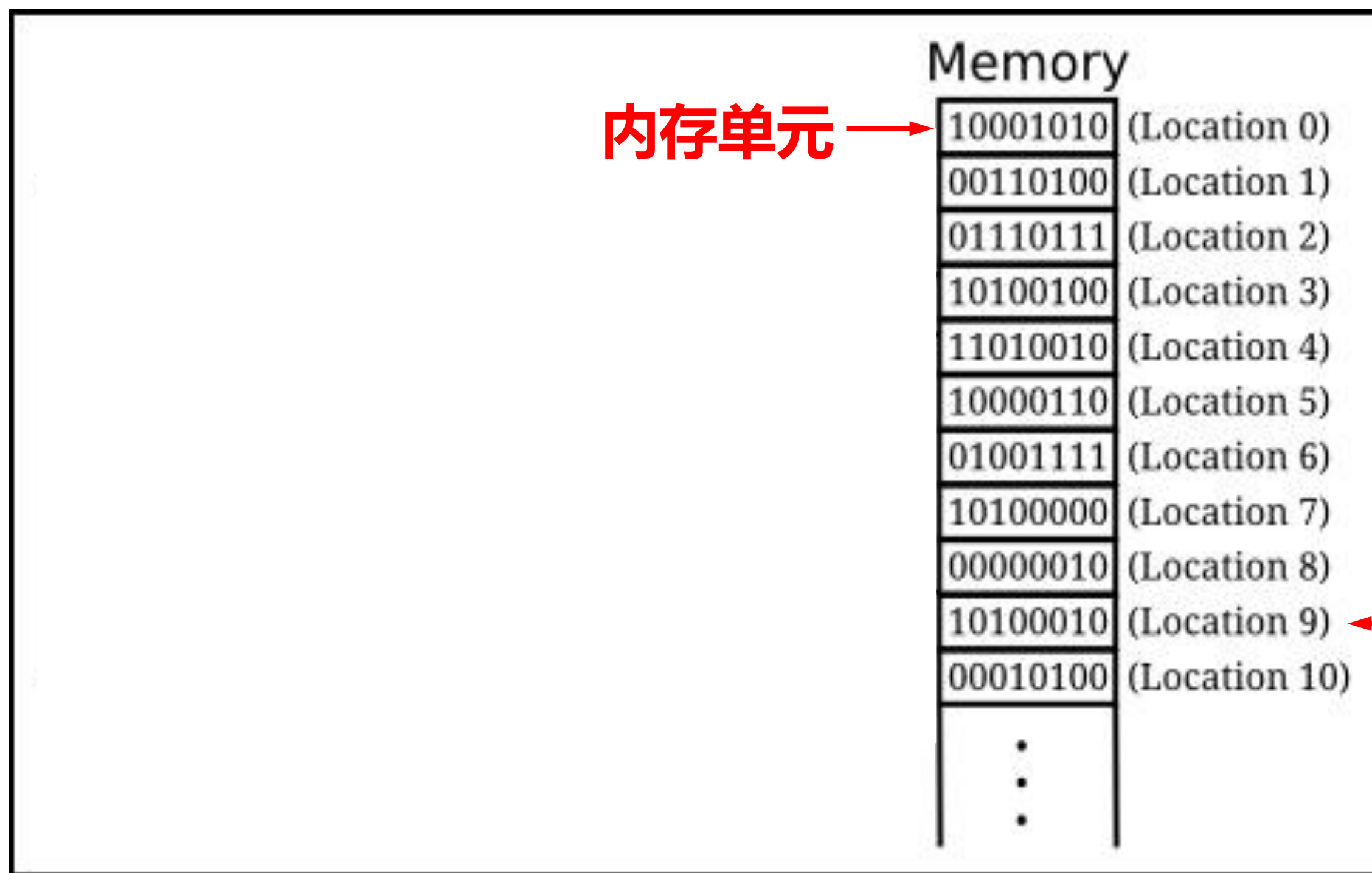


CPU的工作原理

- 从内存中获取并执行一条指令，然后再从内存中获取并执行下一条指令

CPU的工作原理

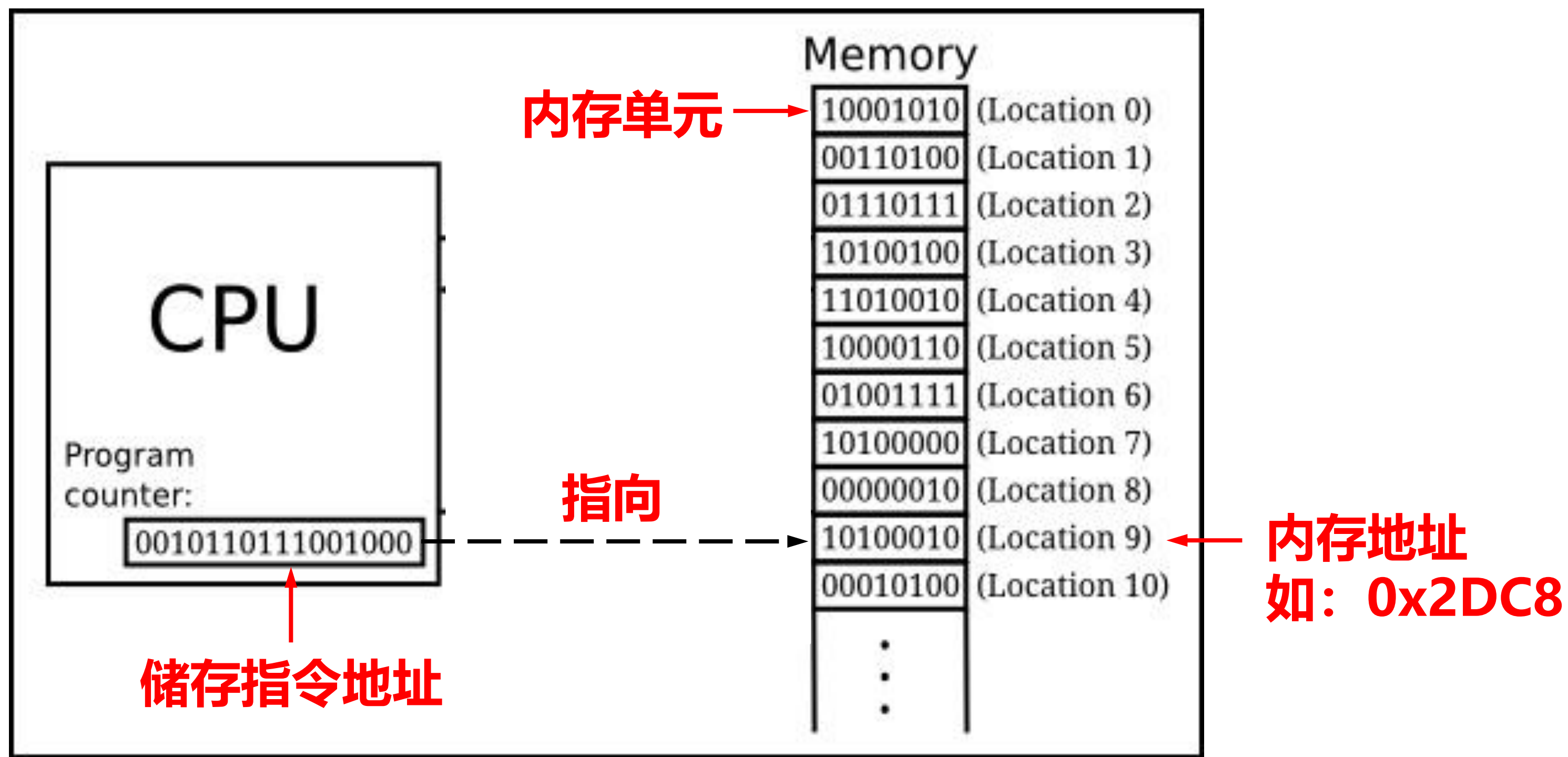
- 内存由许多大小固定 (1字节) 的**内存单元**组成，每个单元有唯一地址



内存地址
如: 0x2DC8

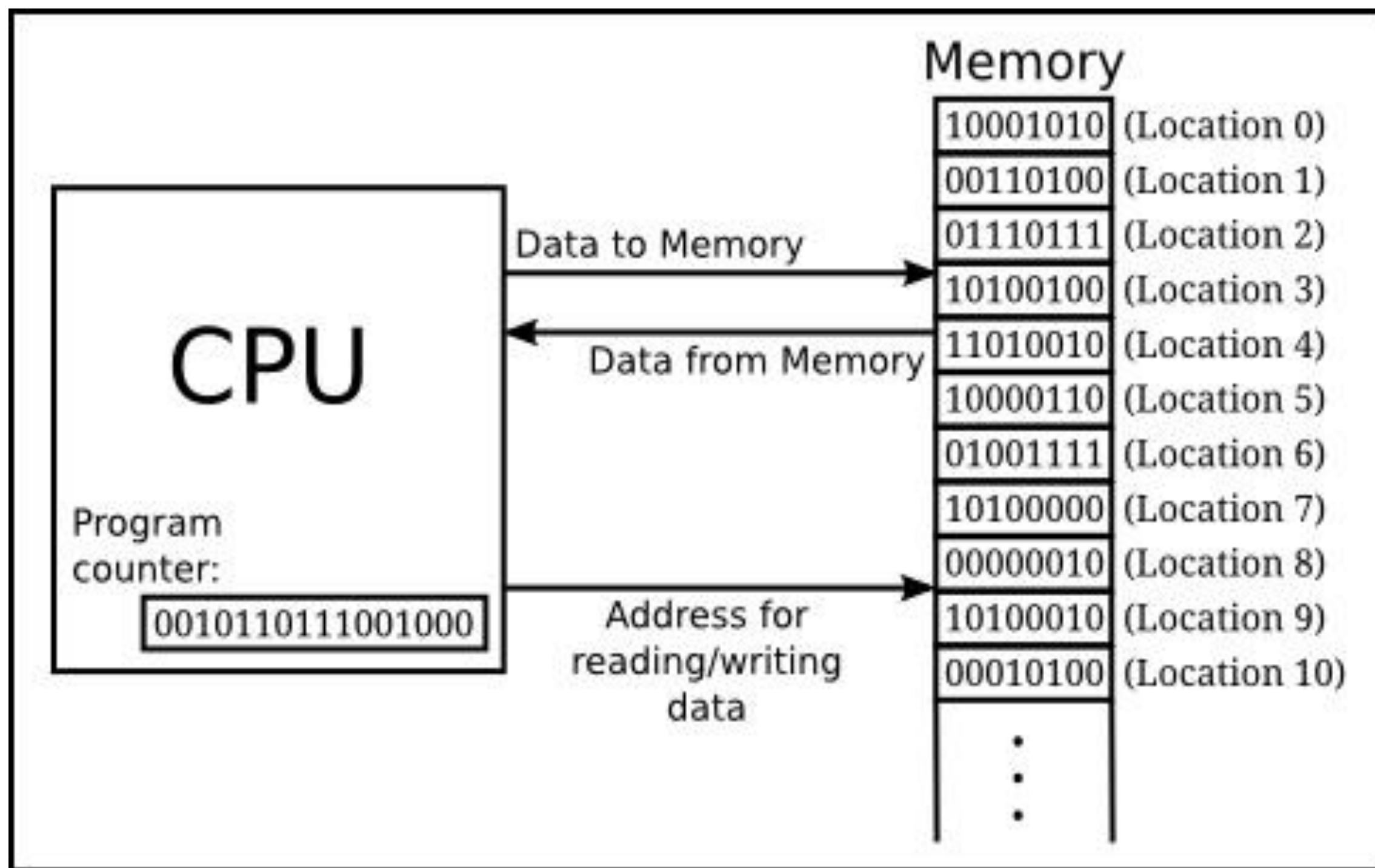
CPU的工作原理

- ❑ 程序计数器 (Program Counter) 存储当前要执行指令所在的内存地址

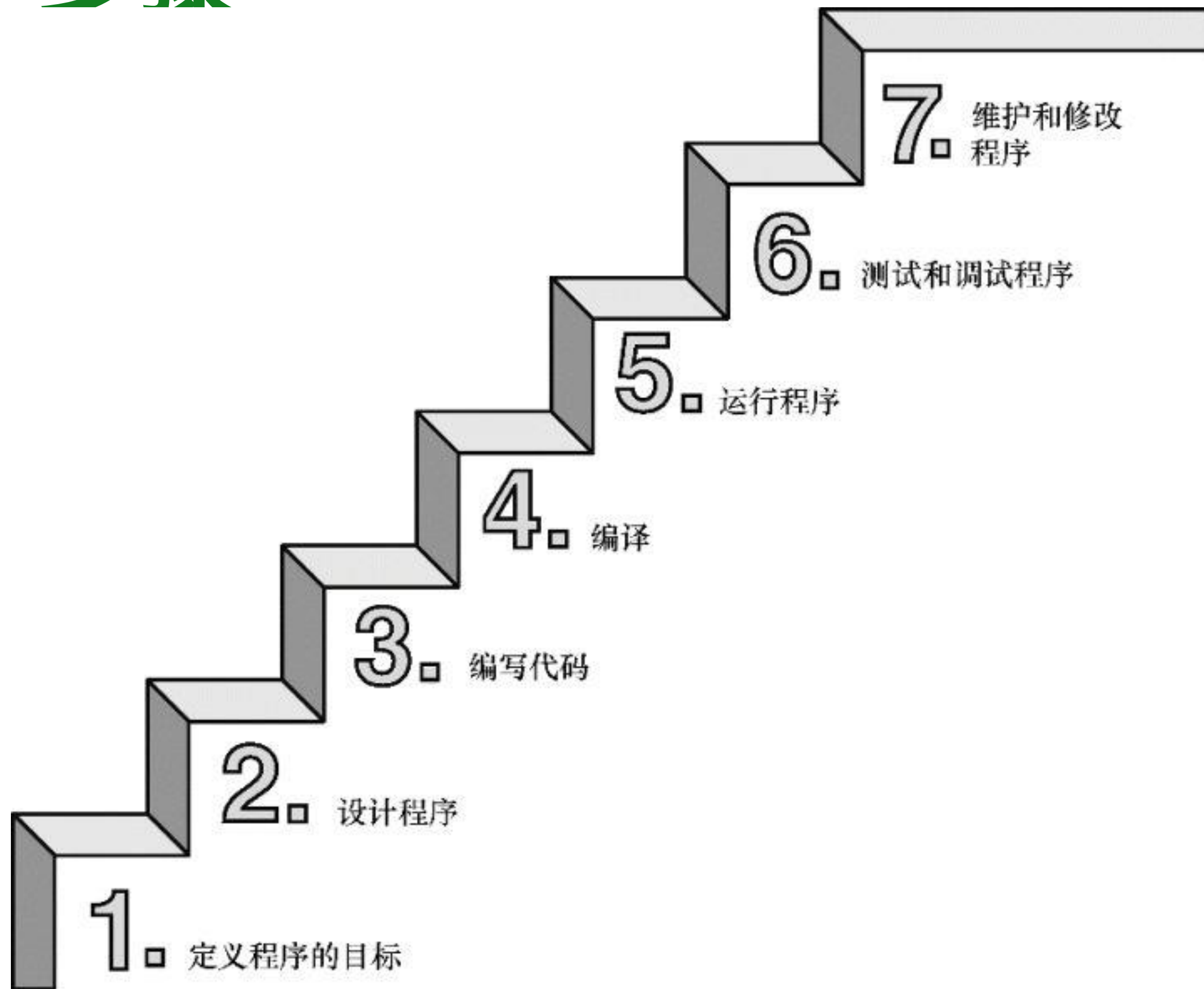


CPU的工作原理

□ CPU指令周期：取指 → 译码 → 执行 → 存储



编程的7个步骤



谢谢